SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Duration: 60 Mins **QUESTIONS: 5** **Total Marks:40**

Annexure A

Case study

Code of Conduct:

I MONISH L [1972260035] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION

Criteria	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Marks/10
Weightage	**25%**	**25%**	**20%**	**20%**	**10%**	**100%**
Q1						
Q2						
Q3						
Q4						

Signature of the Faculty *Total Marks Awarded:*

Annexure A

Case study

QUESTIONS:4 **Total Marks:40**

1. A video conferencing system reports severe call degradation during peak hours. Network logs show 8% packet loss and 180 ms jitter. Identify which protocol layer (TCP/UDP/application) is most likely responsible. Justify your choice with reference to real-time transport requirements and buffering constraints. (BL4) (10)

   Answer: In this scenario the application/transport-layer choice for real-time media is the most likely source of degradation, specifically because the system is evidently using a protocol whose retransmission or congestion-control behavior amplifies delay under loss. Real-time video conferencing should run over UDP (typically via RTP), not TCP, precisely because voice and video frames are time-sensitive: a late packet is worse than a lost one. If TCP were being used for the media stream, 8% packet loss would trigger repeated retransmissions and congestion-window shrinkage, and because TCP enforces strict in-order delivery, a single lost segment would stall the entire buffer until it is recovered, converting packet loss into large added delay -- explaining both the jitter and the perceived degradation. Even with UDP/RTP, 180 ms jitter suggests the receiver's de-jitter buffer is undersized for the network's variability, and 8% loss without forward error correction or a codec that gracefully handles missing frames will cause visible/audible glitches. The fix is to ensure the media path uses UDP-based RTP with adaptive jitter buffering and loss concealment, reserving TCP only for signaling (e.g., SIP/WebRTC control), since TCP's reliability guarantees directly conflict with the low-latency, loss-tolerant needs of real-time transport.

2. DNS resolution for the conferencing domain takes 800 ms on average. Propose a DNS architecture (anycast, pre-fetching, TTL tuning) to bring resolution below 50 ms. Evaluate the trade-offs of your solution under high-availability requirements. (BL4) (10)

   Answer: To bring DNS resolution for the conferencing domain from 800 ms down to under 50 ms, a combination of anycast, prefetching, and TTL tuning is appropriate.

Anycast lets the same DNS server IP be announced from multiple geographically distributed locations, so a client's query is automatically routed to the topologically nearest instance by normal routing, cutting round-trip time and reducing the impact of any single server's load. Prefetching/pre-resolution -- where the client or a nearby recursive resolver refreshes popular records before their TTL expires, or the application resolves the domain proactively before it's needed (e.g., on page load) -- removes the resolution step from the critical path entirely for repeat lookups. TTL tuning involves setting a moderately short TTL so that DNS caches (client, ISP resolver, CDN) refresh often enough to reflect topology or failover changes, but not so short that most requests become cache misses requiring a full recursive lookup. The trade-off is that anycast adds operational complexity and cost (multiple PoPs, consistent zone data), aggressive prefetching increases background query volume and can prefetch records that are never used, and very short TTLs increase load on authoritative servers and reduce the cache-hit benefit; a balanced approach uses anycast for baseline latency, a TTL of roughly 60 to 300 seconds for the failover-sensitive records, and prefetching only for known high-traffic conferencing domains.

3. DNS resolution for the conferencing domain takes 800 ms on average. Propose a DNS architecture (anycast, pre-fetching, TTL tuning) to bring resolution below 50 ms. Evaluate the trade-offs of your solution under high-availability requirements. (BL4) (10)

Answer: Reinforcing the same DNS optimization strategy, the priority in a high-availability deployment is to combine multiple techniques rather than rely on one. Anycast should be the primary latency reducer since it addresses the physical distance component of the 800 ms delay by directing every client to its nearest available server without requiring client-side configuration changes, and it also provides implicit failover if one anycast node fails, traffic simply shifts to the next-nearest node. TTL tuning should be applied selectively: records that rarely change (e.g., static infrastructure) can use longer TTLs to maximize cache hits and minimize authoritative server load, while records tied to failover or load-balancing state should use shorter TTLs so clients pick up changes quickly. Prefetching is best reserved for the conferencing client itself, resolving the domain proactively when the app opens rather than only at call time, so the 800 ms cost is hidden from the user's perceived call-setup latency. The key trade-off under high availability is consistency

versus responsiveness: aggressive caching and long TTLs improve speed and reduce load but slow down failover detection, so critical health-check-driven records should keep TTLs low even at the cost of some added query volume, while non-critical records favor longer TTLs.

4. A stock exchange uses TCP for order submission. During market open, average order-to-ack latency spikes from 1 ms to 40 ms. Using knowledge of TCP's flow control, Nagle's algorithm, and SYN queue limits, identify three probable causes and suggest configuration-level fixes for each. (BL4) (10)

Answer: A latency spike from 1 ms to 40 ms under TCP order-submission load during market open is most plausibly caused by three interacting factors. First, Nagle's algorithm may be delaying small order packets: Nagle's algorithm holds back small writes until either a full segment can be sent or a pending ACK returns, adding up to the receiver's delayed-ACK timer (often ~40 ms) of latency for latency-sensitive, small messages; the fix is to disable Nagle's algorithm by setting TCP_NODELAY on the socket so small order packets are sent immediately. Second, TCP flow control can throttle throughput if the receiver's advertised window shrinks under burst load (many orders arriving simultaneously at market open), forcing the sender to wait for window updates; the fix is to increase the socket receive buffer size and enable TCP window scaling so the window can grow large enough to accommodate bursty traffic without stalling. Third, the SYN queue (backlog) on the exchange's listening socket can overflow when many new connections attempt to open simultaneously at market open, causing SYN retransmissions and connection delays that appear as latency spikes; the fix is to increase the listen backlog size (somaxconn and the socket's backlog parameter) and enable SYN cookies to absorb bursts of new connection attempts without dropping them. Together, disabling Nagle's algorithm, tuning window/buffer sizes, and enlarging the SYN backlog address the three most common causes of TCP latency spikes under bursty, low-latency trading workloads.